



Le rapport du projet d'informatique : Fliptech

Année 2026-2027

Groupe MI5N

Pré-ing Maths-info option physique Cergy

CY TECH

Élèves : Sayhan Bersan et Ould el-moctar Iman

Table des matières

1. Introduction----- pages 2
2. Part 1 : notre organisation pour créer le jeu . ----- pages 3 à 4
3. Part 2 : Les problèmes rencontrés et les solutions apportées avec les résultats .
----- pages 5 à 8 .

INTRODUCTION

Bonjour, nous avons travaillé à 2 pour ce projet. Notre jeu Fliptech consiste à faire un choix : on peut appuyer sur 1 pour piocher une carte, sur 2 pour s'arrêter ou sur 3 pour voir nos cartes actuelles et nos statistiques. Le but est d'avoir le maximum de points (environ 200 points) pour gagner la partie, à condition de ne pas piocher une carte doublon (la même carte numéro), sinon on perd tous les points de la manche en cours. On a choisi ce projet car on aime beaucoup l'idée du jeu de cartes, et stratégiquement, coder un jeu avec plusieurs conditions permettait de mieux gérer la mémoire et d'avoir moins de chances que le jeu crash ou fasse des erreurs.

On a aussi ajouté des améliorations dans ce jeu comme des cartes spéciales, un ordinateur pour jouer contre des bots, des couleurs, des emojis, la possibilité de choisir la taille du paquet de cartes numérotées et pour finir un minuteur de 20 min. En effet, d'après les règles, le jeu dure en moyenne 20 min et cela rajoute une nouvelle condition : si le temps est écoulé ou que la pioche est vide, la partie s'arrête et le gagnant est celui qui possède le maximum de points. Enfin, on a mis un fichier sauvegarde qui enregistre les scores de la partie de manière classée grâce à un algorithme de tri à bulles.

Pour nous organiser, nous avons principalement travaillé sur Discord et WhatsApp. Cela nous a permis de discuter du travail, d'échanger nos idées et de régler les bugs ensemble grâce au partage d'écran.

Au tout début, on ne savait pas vraiment par quoi commencer. Alors, on a pris le temps de bien lire le cahier des charges, les règles du jeu et les critères d'évaluation pour avoir une idée précise de ce qui était attendu.

Nous avons travailler ensemble à chaque étapes .

Partie 1 : notre organisation pour créer le projet .

Dans un premier temps, nous avons mis en place la structure générale du jeu et géré la création du paquet :

En effet, on a commencé par le fichier `fliptech.h` pour définir les constantes comme la taille du paquet, les scores et les couleurs ANSI, et on a donné les définitions des structures essentielles pour le joueur, les cartes et la configuration de la partie.

Après cela, pour le déroulement du jeu dans le fichier `main.c`, on a fait la programmation des menus (choix du mode standard ou personnalisé), de l'enchaînement des manches, de la fin de partie et du classement des scores.

Enfin, on a préparé le paquet dans le fichier `cartes.c` pour créer le paquet et mélanger les cartes.

Dans un deuxième temps, nous avons créé la logique du jeu avec les effets des cartes spéciales :

Premièrement, dans le fichier `déroulement.c`, on a programmé la logique des tours (piocher, s'arrêter) et surtout l'algorithme des cartes spéciales (Stop, Seconde Chance, Trois à la suite).

Deuxièmement, on a intégré la fonction `faire_choix_ordinateur` dans le fichier `ordinateur.c` pour pouvoir jouer contre un ou plusieurs bots.

Troisièmement, on a créé le fichier `affichage.c` avec les fonctions qui affichent les manches, les statistiques et les résultats.

Pour finir, on a fait un fichier `sauvegarde.c` pour l'implémentation du système d'écriture dans un fichier texte afin de conserver l'historique des scores en fin de partie.

Dans une troisième partie, on a essayé d'améliorer l'interface et de la rendre plus ergonomique :

Premièrement, on a essayé d'intégrer des émojis et des couleurs pour rendre le jeu plus coloré et plus vivant.

Deuxièmement, on a créé un fichier makefile en regardant des tutos pour savoir comment ça fonctionne et comment coder ce fichier. La création de ce fichier, en notant la commande make, permet de compiler sans mettre la commande gcc, avec tous les fichiers en .c convertis en .o.

Partie 2 : les problèmes rencontrés et les solutions apportées avec les résultats .

1er problème : Notre premier problème était de ne pas savoir sur quel outil travailler précisément pour commencer à écrire le code et comment réussir à compiler.

Solution : Pour cela, on a discuté et regardé quels étaient les logiciels les plus connus et les plus pratiques. On a décidé de travailler sur Visual Studio. Après l'avoir installé, on a dû le configurer pour coder un jeu en C, car les programmes en C++ utilisent des bibliothèques différentes (comme `<iostream>`). Par la suite, on s'est aperçu que la compilation ne fonctionnait pas. J'ai fait des recherches et on a appris qu'il fallait aller dans la partie "Extensions" de visual studio pour installer une commande de compilation pour le C. Enfin, grâce à des tutos, on a pu le configurer.

Résultat : On peut coder et compiler sans problème.

2^{ème} problème : Lorsque le programme demandait un chiffre (pour le mode de jeu, le nombre de joueurs, notre choix de piocher, s'arrêter ou pour regarder les cartes que l'on possède, et pour enregistrer ou pas nos scores) et que l'utilisateur tapait accidentellement une lettre ou un autre chiffre en dehors des réponses attendu pour continuer, la fonction `scanf` échouait, laissant le caractère dans le buffer. Cela créait une boucle infinie d'affichage de messages d'erreur.

La solution : Nous avons sécurisé chaque `scanf` en vérifiant son retour (if (`saisie_valide != 1`)). Si l'entrée est incorrecte, nous vidons manuellement la mémoire tampon avec une boucle `while` (`getchar() != '\n'`) ; avant de redemander la saisie. Puis, nous avons ajouté des conditions strictes (if / else if) pour rejeter les nombres négatifs ou les choix hors limites : if (`quantite < 0`)

Et aussi pour les saisies de texte (pseudos, noms de fichiers), nous avons limité le nombre de caractères lus (par exemple `%49s` ou `%99s`) pour empêcher tout dépassement de mémoire.

Résultat : Le jeu est plus stable. En cas de lettres ou de chiffres en dehors des réponses attendues, un message d'erreur s'affiche et on redemande au joueur de faire un choix valide pour continuer.

3^{ème} problème : Lors de nos premiers tests, nous avons remarqué que le jeu distribuait exactement la même séquence de cartes à chaque nouvelle partie avec rand ().

La solution : Nous avons ajouté la commande srand((unsigned int)time(NULL)) ; tout au début de notre fichier principal. Cela permet de baser le point de départ de l'aléatoire sur l'horloge interne de l'ordinateur, qui change à chaque seconde. Ensuite, notre algorithme melanger_paquet utilise cet aléatoire pour échanger les positions des cartes.

Résultat : Les tirages sont totalement imprévisibles .

4^{ème} problème : Pendant l'effet de la carte spéciale « Trois à la suite », si la pioche arrivait à court de cartes, le programme tentait de lire une carte en dehors des limites du tableau paquet[]. Cela provoquait un crash immédiat du jeu.

La solution : Nous avons ajouté des sécurités strictes dans le code. Une condition if (*index_pioche >= taille_paquet) a été placée à l'intérieur de la boucle de rafale pour forcer l'arrêt de la pioche avec un break si le paquet est vide. Nous avons également ajouté cette vérification dans la boucle principale du main.c.

Résultat : Il n'y a plus de crash du jeu. Si la pioche est vide, la distribution s'arrête en toute sécurité et la partie se termine proprement.

5^{ème} problème : Avec la carte « Stop », l'ordinateur (ou un joueur humain) pouvait cibler un joueur qui avait déjà perdu ou qui avait déjà quitté la manche. Cela ne servait à rien et provoquait parfois un blocage total du jeu (freeze) pour le bot (l'ordinateur) .

La solution : Nous avons créé un système de « cibles valides ». Avant de permettre le choix, le programme compte le nombre de cibles éligibles avec la variable Cibles_possibles . Avant de faire jouer l'ordinateur, on vérifie avec la condition if (cibles_possibles > 0). S'il n'y a aucune cible valide, la fonction annule l'effet de la carte et utilise simplement la commande return; pour sortir immédiatement de la fonction, ce qui empêche le jeu de planter.

Résultat : Il est désormais impossible de cibler un joueur qui a perdu ou quitté la manche. Cela évite complètement les bugs et les effets de « freeze » pendant la partie.

6^{ème} problème : Dans le mode personnalisé, un joueur pourrait demander à ajouter des cartes d'un certain numéro. Sachant que la mémoire de notre tableau paquet[] a une limite stricte (MAX_PAQUET fixé à 200), cela provoquerait un dépassement de capacité et ferait crasher violemment le programme.

La solution : pour cela avons mis (espace_restant = (MAX_PAQUET – 15) – index ;) dans le fichier cartes.c. Le -15 représente les 6 cartes bonus et les 3 différents cartes spéciales chacun 3 fois copiés (6+9=15) . si la quantité demandée par le joueur dépasse cet espace, le jeu bride automatiquement l'ajout au maximum possible et affiche un message d'alerte rouge pour prévenir que des cartes ont été rejetées.

Résultat : Une sécurité mémoire totale. Le mode personnalisé est indestructible, peu importe les valeurs extrêmes entrées par l'utilisateur.

7^{ème} problème : on arrivait pas à voir correctement les émojis (🎲, 🎴, 🎯) et couleurs sur notre pc sous windows .

La solution : Nous avons ajouté la condition #ifdef _WIN32 dans le fichier main.c pour inclure la bibliothèque <windows.h>. Grâce à cela, nous avons pu utiliser la fonction SetConsoleOutputCP(CP_UTF8) ;, qui force la console Windows à bien lire le format texte des émojis (l'UTF-8).

Résultat : sur notre PC sous Windows, on peut maintenant voir les émojis et couleurs qui s'affichent parfaitement quand on lance le jeu dans le terminal.

8^{ème} problème : À la fin de la partie, les joueurs n'étaient pas classés du meilleur au pire score. De plus, nous voulions un beau podium dans notre fichier de sauvegarde (resultats.txt), mais si nous utilisions les mêmes codes couleurs que pour notre terminal, le fichier texte affichait des symboles illisibles (comme [1 ;33m) car il ne supporte pas les couleurs ANSI.

La solution : Nous avons résolu cela en deux étapes. D'abord, nous avons programmé un Tri à bulles qui compare et échange les places des joueurs pour les classer mathématiquement du plus grand au plus petit score. Ensuite, pour le fichier de sauvegarde, nous avons retiré les codes couleurs et nous avons utilisé des émojis de médailles (, , ) lors de l'écriture.

Résultat : Le programme génère un véritable classement logique basé sur les scores. Le fichier de sauvegarde est propre, esthétique grâce aux médailles, et lisible sur n'importe quel ordinateur sans aucun bug d'affichage.

9^{ème} problème : *On ne savait pas vraiment comment créer un fichier Makefile ni à quoi cela servait. De plus, une fois le fichier codé, la commande n'était pas reconnue par le terminal Windows.

La solution : On a d'abord regardé des tutos pour savoir comment coder le fichier. Pour faire fonctionner la commande, on a utilisé l'outil MinGW. En faisant le raccourci clavier « Windows + R », on a ouvert le dossier C:\msys64\ucrt64\bin. À l'intérieur, on a trouvé le fichier mingw32-make. On l'a copié-collé et on l'a renommé simplement en make pour que le terminal puisse le reconnaître. Ensuite, on a refait un copier-coller de ce fichier pour le nommer make clean afin que cette commande soit aussi reconnue par le système.

Résultat : Lorsqu'on entre make dans le terminal, les fichiers .c sont convertis en .o et ça compile tout seul. Cela facilite grandement la compilation, sans avoir à entrer la longue commande gcc avec tous les fichiers en .c . De son côté, la commande make clean permet d'effacer automatiquement les fichiers .o pour nettoyer le dossier.

